

Dynamic Space-Phase-Time Hypernetwork For Synchronizing Electric Vehicles with Traffic

Signals

Alexander Kaneko

Massachusetts Academy of Math and Science

Advanced STEM with Scientific and Technical Writing

Instructor: Kevin Crowthers, Ph.D.

Worcester, MA. 01605

Abstract

The future of transportation is self-driving electric vehicles. Although they do not pollute the atmosphere with fuel emissions, electric vehicles still consume electricity produced by non-renewable sources such as fossil fuels. By minimizing electricity usage in electric vehicles, fossil fuel emissions will be reduced. Among the various factors affecting electricity consumption in electric vehicles, traffic congestion stands paramount. Therefore, energy consumption can be minimized by improving traffic flow, a factor controlled by traffic signals. One way that traffic signals can be enhanced is by communicating with autonomous vehicles. Specifically, traffic signals could improve adaptability by receiving real-time traffic information from the electric vehicle. In contrast, the vehicle could improve its route planning by knowing what the traffic signal will do ahead of time. Therefore, this proposal's objective is to create a model for a centralized system that synchronizes electric vehicles with traffic signals to decrease network wide energy consumption and traffic congestion. This model built off of a previous traffic signal optimization model which optimized for travel time and adapted it specifically for electric vehicles and minimizing energy consumption. This new model, the previous model, and the status quo was simulated and energy consumption was measured. A t-test computed that my model had an average energy consumption significantly lower than the rest, indicating success. This is relevant for the future as this model can be easily implemented in real-life with a road network of predominantly electric vehicles, which will substantially help the environment and the rising problem of global warming.

Keywords: energy consumption, electric vehicles, traffic signal optimization, route guidance

Acknowledgements

I was advised by Dr. Kevin Crowthers and Mr. Nick Medeiros, who gave instructive guidance during the research process.

Dynamic Space-Phase-Time Hypernetwork For Synchronizing Electric Vehicles with Traffic Signals

Electric cars are the vehicles of the future, as it is predicted that there will be more than 300 million electric vehicles making up more than 60% of new car sales (Paoli, 2022). With this incredible growth, the roads of the future will be populated primarily by these vehicles (Global X Research, 2022). While electric cars do not emit fuel emissions directly as conventional vehicles do, they still consume electricity which is produced in non-sustainable ways. Factories burn fossil fuels such as coal, natural gas, and petroleum in order to generate electric power (Environmental Protection Agency, 2023). During this process, carbon dioxide is released into the environment, which is incredibly harmful to the atmosphere and further fuels global warming.

Impact of traffic on energy consumption

One of the biggest contributors to wasted energy in electric cars is traffic (Jonas, 2022). It was found that electrical consumption was significantly greater during times of high traffic congestion, which means that more energy was wasted (Jonas, 2022). Therefore, optimizing traffic flow would help with reducing energy consumption in electric vehicles, which will also directly decrease carbon emissions.

Improving traffic flow

One possible way that urban engineers can improve traffic flow is by constructing new roads or altering existing ones in order to accommodate the increased demand. Unfortunately, changing the current infrastructure is expensive relative to its benefits (Sanchez et al., 2010). Therefore, the best method to reduce traffic congestion would be to optimize existing traffic

systems that can improve traffic control (Sanchez et al., 2010). The most prominent of such systems are traffic signals, which have an immense influence over the flow of traffic.

Current traffic signals

Many current day traffic signals are fixed-time signals. Fixed-time signals follow predetermined rules and provide the same amount of time for each traffic signal phase, regardless of

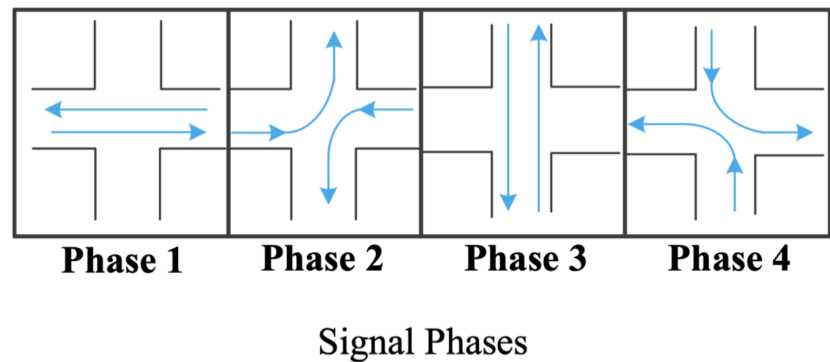


Figure 1. Four phases at an intersection (Li et al., 2015)

current traffic conditions (Practical Engineering, 2019). A traffic signal phase is a group of traffic movements that do not conflict with each other. A normal two-way intersection has four phases as shown in Figure 2, and the traffic signal will cycle through these four phases to clear out any cars waiting at the intersections. Fixed time signals are inexpensive and have acceptable performance during stable traffic conditions. However, since fixed-time signals cannot adapt to real-time traffic demand, they are incredibly inefficient when traffic flow is disturbed by accidents, construction, or big events (Qadri et al., 2020). Traffic signals which do have the ability to adapt are called actuated signals (Practical Engineering, 2019). In order to receive information on the current state of traffic, they often use sensors to detect vehicles.

Optimizing traffic signals

The problem of optimizing the order and timing of traffic signal phases is called the traffic signal optimization problem (TSO). Approaches to the traffic signal optimization problem can be classified into two categories: microsimulation-based and computational intelligence.

Both approaches start with the initial settings for a traffic signal and aim to optimize it by gradually adjusting the order and timings of the traffic phases. This is done recursively, and the adjustment is done based on how well the previous setting performed. Microsimulation-based approaches employ microsimulation techniques for the evaluation performance during this recursive optimization (Qadri et al., 2020). Microsimulation in this context means running a simulation of all the vehicles and traffic signals in the traffic network. One study used a genetic algorithm, a form of machine learning, to help determine how to adjust the settings after microsimulation (Sanchez et al., 2010). The model was found to be effective for highly congested areas of traffic, but this was for only optimizing fixed-time traffic signals (Sanchez et al., 2010). Computational intelligence based-approaches employ an estimation function to evaluate the performance of a certain traffic signal setting without explicitly simulating the network (Qadri et al., 2020). One computational intelligence approach looking at an entire network in real-time used a metaheuristic along with the aid of the PARAMICS software for a multi-objective performance model (Guo et al., 2019). It was found that there was a decrease in average delay in the entire network (Guo et al., 2019).

Gaps in TSO

Although these studies were successful in reducing travel time, they tailored towards regular vehicles without considering energy consumption. Optimizing travel time is different from optimizing energy consumption, since the factors affecting energy consumption are not identical as the factors affecting travel time (Li et al., 2016). The studies involving actuated signals assumed that the only method of receiving live traffic data is through sensors. However, this is not the case for electric vehicles.

Autonomous Electric Vehicles and RGTSO

In the future, electric vehicles will also be autonomous which means that they will drive themselves using artificial intelligence and algorithms (Global X Research, 2022). More importantly, such vehicles will also be able to communicate with external infrastructure, so these vehicles can directly send and receive information to external traffic systems such as traffic signals. This means that traffic signals can be optimized by receiving data from the vehicles to plan optimized routes. The vehicles can then use the data sent from the traffic signals to plan their routes accordingly. This capability motivates research in the simultaneous route guidance and traffic signal timing problem, abbreviated as RGTSO, which my model aims to address. The RGTSO problem is about creating a system where autonomous vehicles are guided upon paths by an external centralized system. The system changes the timings of different traffic phases at signals for the network to have optimal flow.

Previous Work In RGTSO

The problem was first formulated in 1994, where it was solved using an iterative optimization approach after relaxing the problem constraints (Carey & Srinivasan, 1994). Constraints, in this context, refers to the different traffic rules, such as not overflowing the traffic capacity, having a valid path, and stopping at red lights. The constraints were relaxed using the Lagrangian relaxation, which is a method of removing conditions and adding them back onto the objective function. Another effective approach, which also employed Lagrangian relaxation, modeled the traffic network using a space-phase-time hypernetwork and then used integer programming to optimize the routes given and the traffic signal timing (Li et al., 2015). The space-phase-time hypernetwork is a method of representing the route choices and the

traffic signal timings over time. Integer programming is a general method of optimizing an integer function, and was an intuitive choice as Lagrangian relaxation is easily applicable for it. The hypernetwork was effective due to its flexibility and ability to represent complex traffic properties in understandable mathematics (Li et al., 2015). They tested the model on 960 and 480 vehicles in a network of 9 intersections on a three by three grid and found that it was successful in reducing travel time (Li et al., 2015).

Gaps in RGTSO

Li et al., (2015) provides more of a framework rather than a concrete example of it in action. More importantly, the authors only focus on travel time and not on energy consumption like the other TSO models. The simulation software they use is limited and does not have suitable prebuilt packages to analyze electricity consumption and electric vehicles effectively. To build my model, I will expand upon Li et al. 's, (2015) model and solve the simultaneous route guidance and traffic signal timing problem for electric vehicles.

Problem Statement

Traffic causes electric vehicles to waste unnecessary amounts of electricity, which is indirectly increasing the amount of carbon dioxide emitted.

Objective

Obj. 1a: Develop an algorithm that can simultaneously assign routes for vehicles and set traffic signals with given origins and destinations for all the vehicles to optimize specifically for energy consumption. The algorithm performs better than the status quo and the previous RGTSO model.

Obj. 1b: Develop an algorithm that is robust and does not have extreme cases where it performs exceedingly poorly.

Obj. 2: Analyze the runtime of both algorithms to determine its feasibility in real time use.

Section II: Methodology

Role of Student vs. Mentor

I implemented and ran all the algorithms and tests and received helpful guidance and ideas from Dr. Kevin Crowthers and Mr. Nick Medeiros. This project spanned from September to February for a total of 5 months.

Equipment and Materials

All computation was run on a Macbook Air M2, with 24 GB of RAM. The optimization algorithm was implemented in Xcode using the Python programming language. Once a solution was computed, the solution was evaluated using the Simulation of Urban MObility (SUMO) traffic simulator. Afterwards, the algorithm adjusts its solution based on the results of the simulation. An optimal solution is found after iterating, similar to a microsimulation approach to the TSO problem. In order to connect my code to SUMO, the TraCI API was used to synchronize the simulator with my algorithm. Once the results were output into the Terminal, the data was input into a python program in Visual Studio Code. Using the numpy and matplotlib libraries, the data was processed and graphed. In the following sections I will describe how the algorithm was created.

Space-phase-time hypernetwork

As mentioned previously, a space-phase-time hypernetwork is a method of mathematically representing the pathing of the vehicles and the changing of the traffic signals.

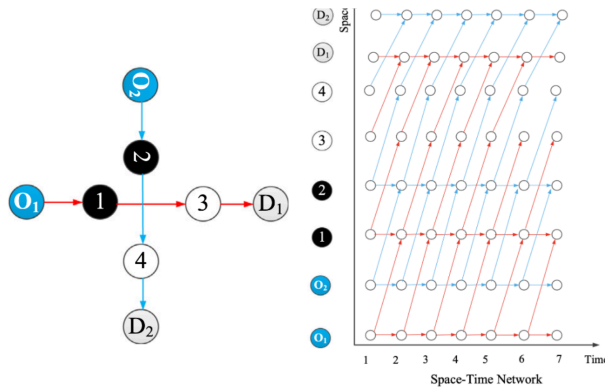


Figure 2. Visual depiction of the space-phase-time network representing the possible routes of the vehicles over time

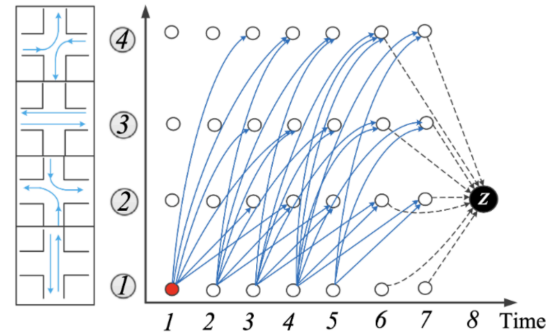


Figure 3. Visual depiction of a space-phase-time hypernetwork representing the possible traffic phase sequences of the traffic signals over time

Its graph-like structure, as shown in Figures 2 and 3 above, opens the door for many techniques which were optimized for graphs, such as dynamic programming. To implement this in the algorithm, I created python classes to create objects for nodes in the space-time network and phase in the phase-time network. As seen in Figures 4 and 5 below, each object has two lists of size “T”, self.Pred and self.lc. Both of these lists represent data for the node at the T different time steps, which allows the algorithm to look at the routes and signals over the course of T seconds.

```
class node:
    global T
    def __init__(self, iswaitingnode, name):
        self.iswaitingnode = iswaitingnode
        self.name = name
        self.links = []
        self.Pred = [None] * T
        self.lc = [999999] * T
    def link(self, other, l, multipliers, SR, phasenum):
        xf = [0] * T
        self.links.append([other, l, multipliers, SR, phasenum, xf])
    def clear(self):
        self.Pred = [None] * T
        self.lc = [999999] * T
```

Figure 4. Python class for nodes in the space-time network

```
class phase:
    global T
    def __init__(self, name, num):
        self.name = name
        self.num = num
        self.costs = [0, 0]
        self.Pred = [None] * T
        self.lc = [999999] * T
    def link(self, P):
        self.costs[P]
```

Figure 5. Python class for phases in the phase-time network

Dynamic programming

The space-phase-time network was the method of representing the problem mathematically, but the technique for actually optimizing it was dynamic programming. Dynamic programming is an optimization technique similar to recursion. However, unlike recursion, dynamic programming stores the values of previously computed subproblems, making it more efficient. Dynamic programming was used for this problem in order to compute the shortest paths for each vehicle. This problem could be seen as vehicles traversing the space-phase-time hypernetwork, and by traveling different nodes there is a cost associated with it. Since the hypernetwork is essentially a graph, computing shortest paths can be seen as a least-cost-path algorithm which can be implemented by using dynamic programming. Figure 6 shows the implementation of the algorithm which computes the optimal path. The algorithm for computing the optimal traffic signal sequence is similar and can be found in Appendix C.

```
def computePath(f, T):
    for n in nodes:
        n.clear()
        f.origin.lc[f.departure] = 0
        LIST = []
        for l in range(T):
            LIST.append([])
        LIST[f.departure].append(f.origin)
        for t in range(f.departure, T - 1):
            for n in LIST[t]:
                if n.isSignalNode == True:
                    if n.lc[t+1] > n.lc[t] + int(not(n == f.destination)):
                        n.lc[t+1] = n.lc[t] + int(not(n == f.destination))
                        n.Pred[t+1] = [n, t]
                        LIST[t + 1].append(n)
                for l in n.links:
                    nID = n
                    nID = l[0]
                    if t + l[1] < T and nID.lc[t] + l[1] + l[2][t] < nID.lc[t + l[1]]:
                        nID.lc[t + l[1]] = nID.lc[t] + l[1] + l[2][t]
                        nID.Pred[t + l[1]] = [nID, t]
                        LIST[t + l[1]].append(l[0])

    #PRINTS THE OPTIMAL ROUTE
    print("----- OPTIMAL ROUTE -----")
    currentnode = f.destination
    t = T-1
    while t > f.departure:
        f.path.insert(0, (currentnode, t))
        print( (currentnode.name, t) )
        if currentnode.Pred[t] == None:
            break
        if not(currentnode.Pred[t][0] == currentnode):
            currentnode.Pred[t][0].links[Linkmap[ currentnode.Pred[t][0].name + currentnode.name ][5][currentnode.Pred[t][1]] += 1
        currentnode, t = currentnode.Pred[t][0], currentnode.Pred[t][1]
    f.path.insert(0, (f.origin, f.departure))

    print([f.origin.name, f.departure])
    print(f.path)
```

Figure 6. Dynamic Programming algorithm for computing optimal sequence

Simulation Software

Once the paths and the traffic signal sequence was obtained, the solution was tested in a simulation software. Choosing the right software was an important decision, so I employed a decision matrix to decide. This can be found in Appendix B, which shows the criteria which I used to determine that the SUMO simulation software was the most effective. This is due to its pre-built functions for analyzing energy consumption and electric vehicles, which enabled me to analyze them more efficiently. In order to use the software, the TraCI API was used to connect to the simulation software in the python script. Figure 7 shows the code for doing so.

```
if __name__ == "__main__":
    options = get_options()

    # check binary
    if options.nogui:
        sumoBinary = checkBinary('sumo')
    else:
        sumoBinary = checkBinary('sumo-gui')

    sumoBinary = checkBinary('sumo')

    # traci starts sumo as a subprocess and then this script connects and runs

    for i in range(n):
        traci.start([sumoBinary, "-c", "demo.sumocfg",
                      "--tripinfo-output", "tripinfo.xml"])
        run(True)

    hugereset()
    for i in range(n):
        traci.start([sumoBinary, "-c", "demo.sumocfg",
                      "--tripinfo-output", "tripinfo.xml"])
        run(False)
```

Figure 7. Connecting to SUMO

Once a feasible solution was obtained, the average energy consumption was recorded. To make the solution more effective, the algorithm used the results of this simulation to slightly adjust the paths and sequences in a way that decreases energy consumption. The process of adjusting solutions and simulating was completed 30 times to eventually find the most effective solution.

Statistical Tests

This model for optimizing energy consumption was tested against fixed-time signals and the previous RGTSO model for travel time. 50 sample cases were tested with random levels of congestion and varying origins and destinations. The average energy consumption was computed for both models and were compared using a t-test for mean difference.

This model vs fixed-time signals

For each of the 50 different test cases, the difference in average electricity consumption in Wh was computed. A t-test for mean difference was used after finding the mean and standard deviation, leading us to compute the p-value.

This model vs previous RGTSO model

Similarly, for every test case, the difference in average electricity consumption in Wh was computed. A t-test for mean difference was used after finding the mean and standard deviation, leading us to compute the p-value.

Section III: Results

Three models were tested: our model optimizing for energy consumption, fixed traffic signals, and the previous RGTSO model which were optimized for travel time. The traffic conditions were randomized for all 50 test cases, and the SUMO simulation software recorded data on the electric vehicles.

Analyzing electricity consumption

In this context, the success of a model is measured by the average electricity consumption. This was computed by SUMO for all three models, for each of the 50 test cases.

This model vs fixed-time signals

Figure 8 visually compares the performance of this model to fixed-time traffic signals. The blue dots represent the average electricity consumption of fixed-time signals, while the red dots depict the average electricity consumption of our model. For each test case, a vertical red line was drawn if our model performed better (smaller average consumption), otherwise a vertical blue line was drawn.

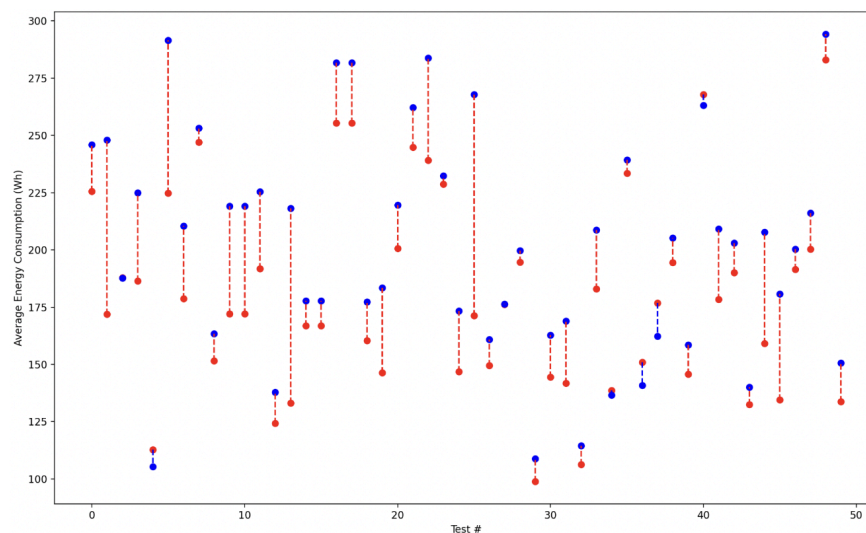


Figure 8. Graph comparing the average energy consumption of our model (red) to the status quo (blue) over the 50 test cases. For each test case, a red line is drawn if the our model performed better, otherwise a blue line is drawn.

The mean and standard deviation for both models is shown numerically in Table 1. Because the data is paired and there are more than 30 test cases, a 1-sample t-test for mean difference can be performed to determine if there is a statistical significance. The t-test yields a p-value of $9.67 \cdot 10^{-9}$.

Table 1: Statistical Results

	This mode	Status quo	Mean difference
Average electricity Consumption (Wh)	179.384	201.543	22.158
Standard deviation	43.585	48.880	23.383

This model vs previous RGTSO model

Figure 9 visually compares the performance of this model to the previous RGTSO model which is optimized for travel time. The blue dots represent the average electricity consumption of the previous RGTSO model, while the red dots depict the average electricity consumption of our model. For each test case, a vertical red line was drawn if our model performed better (smaller average consumption), otherwise a vertical blue line was drawn.

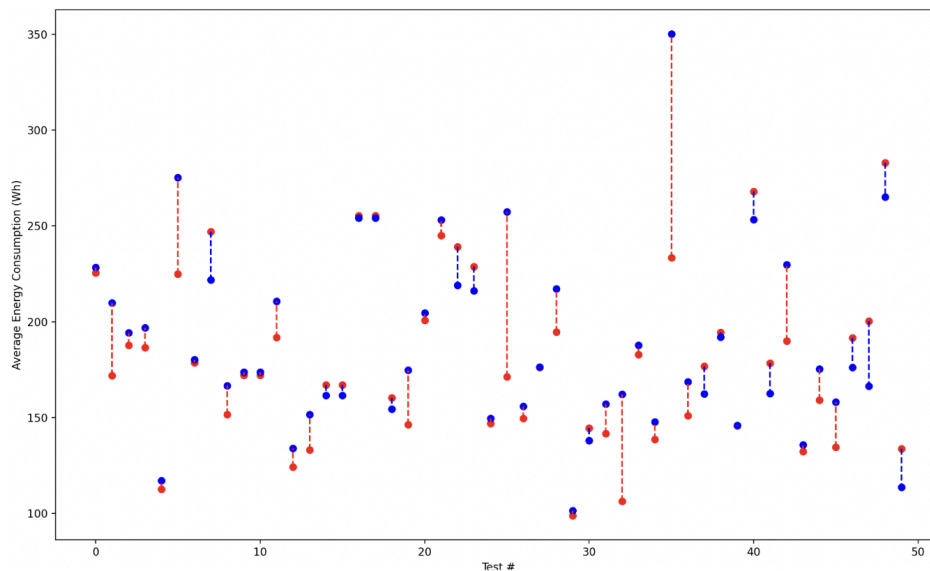


Figure 9. Graph comparing the average energy consumption of our model (red) to the previous RGTSO model (blue) over the 50 test cases. For each test case, a red line is drawn if the our model performed better, otherwise a blue line is drawn.

This mean and standard deviation for both models is shown numerically in Table 2.

Because the data is paired and there are more than 30 test cases, a 1-sample t-test for mean

difference can be performed to determine if there is a statistical significance. The t-test yields a p-value of *0.0139.

Table 2: Statistical Results

	This mode	Li's model	Mean difference
Average electricity Consumption (Wh)	179.384	187.876	8.490
Standard deviation	43.585	47.924	26.477

Analyzing computation time

The runtime of our model's algorithm for each of 50 test cases was measured. The data is shown visually in Figure 10. The average runtime was 3.238 seconds with a standard deviation of 0.412 seconds.

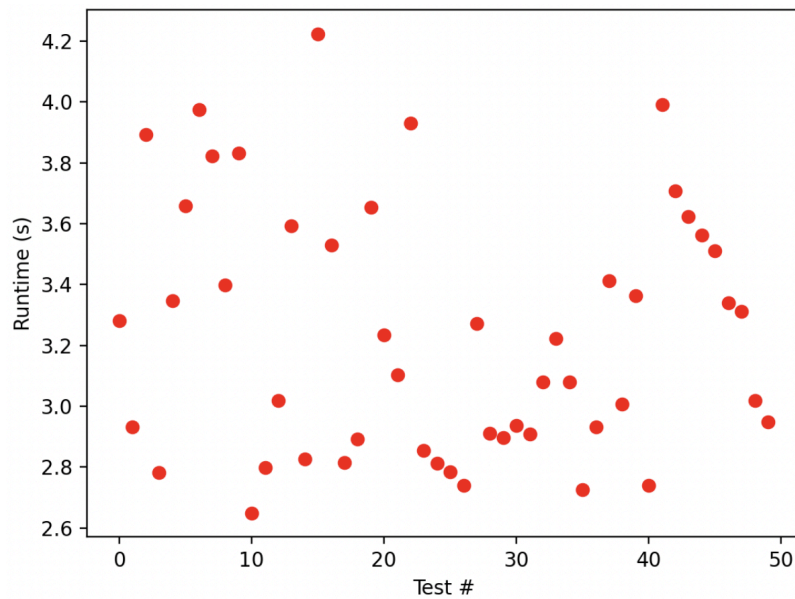


Figure 10. Graph depicting the runtime of the algorithm over the 50 test cases.

Section IV: Discussion

In order to reduce the average energy consumption of electric vehicles in a traffic network, this model aimed to solve the simultaneous route guidance and traffic signal timing problem for electric vehicles. An algorithm was developed and tested with respect to the status quo and the previous RGTSO model which optimized for travel time.

This new model vs standard and previous model

When compared to the status quo, our model was on average 22 Wh more efficient, which is a reduction of 11%. (Table 1). Since the model was able to significantly reduce the average energy consumption, Objective 1a was achieved. Our model also performed better than the previous RGTSO model by 8.490, which is a reduction of 4.5% (Table 2). Therefore, our model is optimized specifically for electricity, confirming that optimizing for travel time is different than optimizing for energy consumption.

Robustness of model

As shown in Figures 8 and 9, our model performed consistently better than both models because of the presence of red lines. Although there are some blue lines, representing the cases where our model performed worse, these are not common and are never greater than 50 Wh in size. Therefore, our model is robust as it does not have any extreme cases where it performs significantly worse, meaning that it achieves Objective 1b. Furthermore, many of the red lines are considerably large, which means that it is common for our model to perform exceedingly better.

Feasibility of application

Finally, my algorithm took on average only 3.238 seconds to run, taking less than 4.2 seconds on its worst run (Figure 10). Therefore, this model is feasible to be used in real time due to the efficiency of the algorithm, satisfying Objective 2. This was a challenge to overcome, since initially the program was very slow and took minutes to run. I overcame this obstacle by simplifying code and optimizing time complexity, which shows how important efficient algorithms are in practical applications. Furthermore, the algorithm can be even faster with more powerful computing. The tests were run on a Macbook Air, so utilizing supercomputers could potentially reduce the computation to milliseconds.

Statistical Tests

To compare our model to the status quo and previous RGTSO model, 1-sample t-tests for mean difference were used. This test was used because the data was naturally paired, so comparing the mean difference would show how the models differed for each individual test case. There was statistical significance in both tests, due to the p-values being less than our significance level of 0.05. One possible confounding variable is that the testing data was randomly generated, which may not be completely representative of real-life traffic flow. However, this fact does not prove that the model is ineffective but just means that it should be calibrated with real-time data when used in the real-world.

Previous studies

This work confirms many of the previous studies which found that traffic signal optimization models reduced travel time, especially for higher levels of congestion (Sanchez et al., 2010). However, my model differs due to its focus on energy consumption and electric vehicles, which is becoming increasingly relevant as the future of AI and automation comes. The idea of using the SUMO simulation software along with dynamic programming and the hypernetwork significantly improved the performance of the previous model, making it more viable to be in use for the real-world (Li et al., 2015).

Implications and applications

Due to the incredibly fast runtime of the algorithm, the model is feasible to be applied in real time traffic control. Electric vehicle companies and transportation departments can work together and apply this model to decrease our carbon footprint and improve traffic flow. Additionally, the fact that optimizing for energy consumption is different than optimizing for travel time implies urban engineers must take energy efficiency in their work. Although certain traffic structures may help optimize travel time, shifting the focus to energy consumption is critical in today's day and age of global warming.

Future Research

The major assumption which this model used was that every vehicle was electric and could be guided by an external system. While this will be accurate in the future, this may not be currently viable as there may be some traffic environments which have both electric and regular

vehicles. In such a case, the paths of the regular vehicles would not be guided, so the traffic optimization algorithm would need to account for that. Doing further research on adapting this model for a mixed traffic environment would make it more accurate for real-world traffic conditions.

In addition, looking at a bigger network would help make it more effective in a larger urban environment. Although it will be more computationally intensive, looking at larger networks will improve performance as planning over a larger area allows for more flexibility. The dynamic programming approach is scalable, which makes this a feasible idea to investigate (Li et al., 2015).

Section V: Conclusion

The objective of this study was to develop an algorithm that can simultaneously assign routes for vehicles and set traffic signals with given origins and destinations for all the vehicles to optimize energy consumption. Through a creative mix of integer programming, a space-phase-time hypernetwork, dynamic programming, and the SUMO simulation software, the model was effective. Specifically, it was able to reduce energy consumption by 11% when compared to the current-day traffic system, which is significant as verified by a 1-sample mean difference t-test. By improving upon a previous study, this model was indeed successful in making vehicles better for the environment. As our society is increasingly facing the consequences of global warming, it is essential that we use the most of our existing circumstances and create new technologies that can reverse the damage that we have done. Therefore, by using our existing traffic signal system and pairing it with the new electric

vehicles, we can guarantee that our transportation systems will be doing its part in saving our planet.

Section VI: References

Barth, & Boriboonsomsin, K. (2008). Real-World Carbon Dioxide Impacts of Traffic Congestion.

Transportation Research Record, 2058(1), 163–171. <https://doi.org/10.3141/2058-20>

Boyles, S. D., N. E. Lownes, and A. Unnikrishnan. (2022) Transportation Network Analysis, Volume I, Version 0.90.

Carey, M., Srinivasan, A., 1994. Solving a class of network models for dynamic flow control. European Journal of Operational Research 75 (1), 151–170.

Downs, A. (2004, January 1). Traffic: Why It's Getting Worse, What Government Can Do.

Brookings. Retrieved November 8, 2022, from

<https://www.brookings.edu/research/traffic-why-its-getting-worse-what-government-can-do/>

Environmental Protection Agency. (2023). Sources of Greenhouse Gas Emissions. EPA.

<https://www.epa.gov/ghgemissions/sources-greenhouse-gas-emissions>

Global X Research. (2022). *The Future of Transportation is autonomous & Electric*. Global X ETFs.

Retrieved January 25, 2023, from

<https://www.globalxetfs.com/future-of-transportation-is-autonomous-electric/>

Guo, Kong, Y., Li, Z., Huang, W., Cao, J., & Wei, Y. (2019). A model and genetic algorithm for area-wide intersection signal optimization under user equilibrium traffic. *Mathematics and Computers in Simulation*, 155, 92–104.

<https://doi.org/10.1016/j.matcom.2017.12.003>

Jonas, Hunter, C. D., & Macht, G. A. (2022). Quantifying the Impact of Traffic on Electric Vehicle Efficiency. *World Electric Vehicle Journal*, 13(1), 15–.

<https://doi.org/10.3390/wevj13010015>

Li, Mirchandani, P., & Zhou, X. (2015). Solving simultaneous route guidance and traffic signal optimization problem using space-phase-time hypernetwork. *Transportation Research. Part B: Methodological*, 81, 103–130. <https://doi.org/10.1016/j.trb.2015.08.011>

Li, Stanula, P., Egede, P., Kara, S., & Herrmann, C. (2016). Determining the Main Factors Influencing the Energy Consumption of Electric Vehicles in the Usage Phase. *Procedia CIRP*, 48, 352–357. <https://doi.org/10.1016/j.procir.2016.03.014>

Paoli, L. (2022). Electric vehicles. IEA. <https://www.iea.org/reports/electric-vehicles>

Practical Engineering. (2019). How Do Traffic Signals Work? YouTube. Retrieved September 18, 2022, from <https://www.youtube.com/watch?v=DP62ogEZgkI>

Qadri, Gökçe, M. A., & Öner, E. (2020). State-of-art review of traffic signal control methods: challenges and opportunities. *European Transport Research Review*, 12(1), 1–23. <https://doi.org/10.1186/s12544-020-00439-1>

Retallack, & Ostendorf, B. (2019). Current Understanding of the Effects of Congestion on Traffic Accidents. *International Journal of Environmental Research and Public Health*, 16(18), 3400–. <https://doi.org/10.3390/ijerph16183400>

Sanchez-Medina, Galan-Moreno, M. J., & Rubio-Royo, E. (2010). Traffic signal optimization in “La Almozara” district in saragossa under congestion conditions, using genetic algorithms, traffic microsimulation, and cluster computing. *IEEE Transactions on Intelligent Transportation Systems*, 11(1), 132–141. <https://doi.org/10.1109/TITS.2009.2034383>

Wang, & Xu, Y. (2011). Estimating O-D travel time matrix by Google Maps API: implementation, advantages, and implications. *Annals of GIS*, 17(4), 199–209. <https://doi.org/10.1080/19475683.2011.625977>

Section VII: Appendices

Appendix A: Limitations and Assumptions

Limitations:

1. External funding was not available.
2. Computational power was limited

Assumptions:

1. A randomly generated traffic network is representative
2. All vehicles are electric
3. All vehicles are connected and are being guided by the centralized system

Appendix B: Engineering Matrix: The Comparison Between Possible Simulation Softwares

Decisions Matrix

Criteria	DTALite	NetLogo	SUMO
Flexible			
Microscopic Simulation			
Heavy loads			
*Models electric vehicles			

- DTALite: Mesoscopic traffic simulation used by the previous RGTSO model. Did not have libraries specifically designed for electric vehicles, but was able to handle highly complex algorithms.
- Design B: NetLogo: Agent-based simulation software. Effective with modeling microscopically but could not easily support the complex algorithm which was used in this study.
- Design C: SUMO: Flexible simulation softwares with pre-built functions for electric vehicles. Incredibly fast performance and could handle heavy computation.

Appendix C: Code for algorithm and API

```

from inputdata import *
import time

'''SPACE TIME PROGRAM '''

''' Globals'''
#Time limit
T = 100
kjam = 4
n = 30
stepsize = 1/(100* n+1)
TIMES = []
CONSUMPTIONold = []
CONSUMPTIONour = []

''' Classes'''

#nodes of the graph
class node:
    global T
    def __init__(self, iswaitingnode, name):
        self.issignalNode = iswaitingnode
        self.name = name
        self.links = []
        self.Pred = [None] * T
        self.lc = [999999] * T
    def link(self, other, l, multipliers, SR, phasenum):
        xf = [0] * T
        electricity = [0] * T
        self.links.append([other, l, multipliers, SR, phasenum, xf, electricity])
    def clear(self):
        self.Pred = [None] * T
        self.lc = [999999] * T

#Vehicle
class vehicle:
    def __init__(self, o, d, rf, name):
        self.origin = o
        self.destination = d
        self.departure = rf
        self.path = []
        self.name = name

'''Objects'''
#Nodes

```

```

O1 = node(False, "O1")
N1 = node(True, "N1")
N3 = node(False, "N3")
D1 = node(True, "D1")

O2 = node(False, "O2")
N2 = node(True, "N2")
N4 = node(False, "N4")
D2 = node(True, "D2")

nodes = [O1, N1, N3, D1, O2, N2, N4, D2]
origins = [O1, O2]
destinations = [D1, D2]
travelnodes = [O1, N3, O2, N4]
signalnodes = [N1, N2]

#Lagrange multipliers
Ut = [ [0] * T] * len(travelnodes) #travel link multipliers
Lm = [ [1] * T] * len(signalnodes) #signal link multipliers

#Inflow capacity
SR = [ [1] * T] * 6

#Links
O1.link(N1, 20, Ut[0], SR[0], None)
N1.link(N3, 10, Lm[0], SR[1], 0)
N3.link(D1, 20, Ut[1], SR[2], None)

O2.link(N2, 20, Ut[2], SR[3], None)
N2.link(N4, 10, Lm[1], SR[4], 1)
N4.link(D2, 20, Ut[3], SR[5], None)

#linkmap
linkmap = { "O1N1" : 0, "N1N3": 0, "N3D1": 0,
            "O2N2" : 0, "N2N4" : 0, "N4D2": 0}

#Vehicles
odmap = { O1 : D1, O2 : D2}
originmap = {1 : O1, 2: O2}

origins = []
departures = getDepartures()

orig = getOrigins()
for j in orig:

```

```

origins.append(originmap[j])

vehicles = []
for i in range(len(origins)):
    vehicles.append(vehicle(origins[i], odmap[origins[i]], departures[i], "v" + str(i)))

''' Algorithm'''
def computePath(f, T):
    for n in nodes:
        n.clear()
    f.origin.lc[f.departure] = 0
    LIST = []
    for i in range(T):
        LIST.append([])
    LIST[f.departure].append(f.origin)
    for t in range(f.departure, T - 1):
        for n in LIST[t]:
            if n.issignalNode == True:
                if n.lc[t+1] > n.lc[t] + int(not(n == f.destination)):
                    n.lc[t+1] = n.lc[t] + int(not(n == f.destination))
                    n.Pred[t+1] = [n,t]
                    LIST[t + 1].append(n)
            for l in n.links:
                nlU = n
                nlD = l[0]
                if t + l[1] < T and nlU.lc[t] + l[1] + l[2][t] < nlD.lc[t + l[1]]:
                    nlD.lc[t + l[1]] = nlU.lc[t] + l[1] + l[2][t]
                    nlD.Pred[t + l[1]] = [nlU, t]
                    LIST[t + l[1]].append(l[0])

    currentnode = f.destination
    t = T-1
    while t > f.departure:
        f.path.insert(0, [currentnode, t])
        #print( [currentnode.name, t] )
        if currentnode.Pred[t] == None:
            break
        if not(currentnode.Pred[t][0] == currentnode):
            currentnode.Pred[t][0].links[linkmap[ currentnode.Pred[t][0].name +
currentnode.name]][5][currentnode.Pred[t][1]] += 1
            currentnode, t = currentnode.Pred[t][0], currentnode.Pred[t][1]
        f.path.insert(0, [f.origin, f.departure])

    #print([f.origin.name, f.departure])
    #print(f.path)

```

```

''' ----- '''
''' ----- '''
''' ----- '''

```

```
'''PHASE TIME PROGRAM '''
```

```
'''Globals '''
```

```
gmin = 7
```

```
gmax = 20
```

```
'''Classes '''
```

```
#Signal phase
```

```
class phase:
```

```
    global T
```

```
    def __init__(self, name, num):
```

```
        self.name = name
```

```
        self.num = num
```

```
        self.costs = [0, 0]
```

```
        self.Pred = [None] * T
```

```
        self.lc = [999999] * T
```

```
    def link(self, P):
```

```
        self.costs[P]
```

```
    def clear(self):
```

```
        self.Pred = [None] * T
```

```
        self.lc = [999999] * T
```

```
#Phases
```

```
P1 = phase("P1", 0)
```

```
P2 = phase("P2", 1)
```

```
Z = [999999, None]
```

```
phases = [P1, P2]
```

```
#Mapping matrix
```

```
M = [ [1, 0], [0, 1] ]
```

```
'''Algorithm'''
```

```
#Calculating arc costs
```

```
def cost(mk, mk1, t, h):
```

```
    sum = 0
```

```
    for i in range(t, h):
```

```
        for s in signalnodes:
```

```

        sum += (s.links[0][2][i] * s.links[0][3][i] * M[mk.num][s.links[0][4]])
    return sum

sequence = []
def reset():
    global sequence
    sequence = []
    for p in phases:
        p.clear()
    P1.lc[0] = 0
def hugereset():
    for N in signalnodes:
        for l in N.links:
            l[2] = [1] * T
            l[3] = [1] * T
    for N in travelnodes:
        for l in N.links:
            l[2] = [0] * T
            l[3] = [1] * T

#Finding the sequence
def computeSequence(T):
    for t in range(T):
        for mk in phases:
            if t + gmin > T:
                if Z[0] > mk.lc[t]:
                    Z[0] = mk.lc[t]
                    Z[1] = [mk, t]
            else:
                for mk1 in phases:
                    if not(mk1 == mk):
                        for dt in range(gmin, gmax):
                            if dt + t < T:
                                c = cost(mk, mk1, t, t + dt)
                                if mk1.lc[t+dt] > mk.lc[t] + c:
                                    mk1.lc[t+dt] = mk.lc[t] + c
                                    mk1.Pred[t+dt] = [mk, t]

#PRINTS THE OPTIMAL SEQUENCE
print("---- OPTIMAL SEQUENCE----")
t = Z[1][1]
currentphase = Z[1][0]
while t >= 0:
    sequence.insert(0, [currentphase.name, t])

```

```

    #print( [currentphase.name, t] )
    if currentphase.Pred[t] == None:
        break
    currentphase,t = currentphase.Pred[t][0],currentphase.Pred[t][1]

''' -----'''
''' -----'''
''' -----'''

'''API STUFF'''

#!/usr/bin/env python

import os
import sys
import optparse

# we need to import some python modules from the $SUMO_HOME/tools directory
if 'SUMO_HOME' in os.environ:
    tools = os.path.join(os.environ['SUMO_HOME'], 'tools')
    sys.path.append(tools)
else:
    sys.exit("please declare environment variable 'SUMO_HOME'")

from sumolib import checkBinary # Checks for the binary in environ vars
import traci

def get_options():
    opt_parser = optparse.OptionParser()
    opt_parser.add_option("--nogui", action="store_true",
                          default=False, help="run the commandline version of sumo")
    options, args = opt_parser.parse_args()
    return options

'''sumo_cmd = "sumo -c demo.sumocfg"
sumo_proc = subprocess.Popen(sumo_cmd.split())
traci = traci.connect(port=8813)'''

def addvehicle(v):
    traci.route.add(v.name, [v.origin.name, v.destination.name])
    traci.vehicle.add(v.name, v.name, typeID="ev", depart=v.departure)

phasemap = {"P1": 0, "P2": 2}

```

```
reversemap = {"P1": 2, "P2" : 0}
```

```
def idlink(n1, n2):
```

```
    id = ""
```

```
    if not(n2 in destinations):
```

```
        id += n1.name
```

```
    if not(n1 in origins):
```

```
        id += n2.name
```

```
    return id
```

```
# contains TraCI control loop
```

```
def run(isourmodel):
```

```
    st = time.time()
```

```
    reset()
```

```
    step = 0
```

```
    traffic_light_ids = traci.trafficlight.getIDList()
```

```
    for v in vehicles:
```

```
        addvehicle(v)
```

```
        computePath(v, T)
```

```
    computeSequence(T)
```

```
    i = 0
```

```
    ceiling = 0
```

```
    totalelectricity = 0
```

```
    while traci.simulation.getMinExpectedNumber() > 0 and step < T:
```

```
        traci.simulationStep()
```

```
        #USE THE PHASE SEQUENCE
```

```
        if step == ceiling and i < len(sequence) - 2:
```

```
            dur = sequence[i+1][1] - sequence[i][1]
```

```
            traci.trafficlight.setPhase("N1", phasemap[sequence[i][0]])
```

```
            traci.trafficlight.setPhaseDuration("N1", dur)
```

```
            traci.trafficlight.setPhase("N2", phasemap[sequence[i][0]])
```

```
            traci.trafficlight.setPhaseDuration("N2", dur)
```

```
            ceiling = sequence[i+1][1]
```

```
            i+=1
```

```
        #GETS DATA ON SR
```

```
        for N in nodes:
```

```
            for l in N.links:
```

```
                edgeid = idlink(N, l[0])
```

```
                #print(edgeid)
```

```
                volume = traci.edge.getLastStepVehicleNumber(edgeid)
```

```
                l[3][step] = (volume - kjam)/T
```

```
            if isourmodel:
```

```
                l[6][step] = traci.edge.getElectricityConsumption(edgeid)
```

```

#GETS ELECTRICITY CONSUMPTION
vehicle_ids = traci.vehicle.getIDList()
for vehicle_id in vehicle_ids:
    totalelectricity += traci.vehicle.getElectricityConsumption(vehicle_id)

step += 1

#CALCULATING AVG TRAVEL TIME
vehicle_ids = traci.vehicle.getIDList()
totaltime = 0
total_travel_time = 0
num_vehicles = len(vehicle_ids)
for vehicle_id in vehicle_ids:
    travel_time = traci.vehicle.getAccumulatedWaitingTime(vehicle_id)
    total_travel_time += travel_time
avg_travel_time = total_travel_time / num_vehicles
TIMES.append(avg_travel_time)

#CALCULATING AVERAGE ENERGY CONSUMPTION
if isourmodel:
    CONSUMPTIONour.append(totalelectricity/num_vehicles)
else:
    CONSUMPTIONold.append(totalelectricity/num_vehicles)
# Update Lagrange Multipliers's
for N in nodes:
    for l in N.links:
        for t in range(0, T):
            if isourmodel:
                l[2][t] = l[2][t] + stepsize * (l[5][t] - l[3][t] + l[6][t]/totalelectricity)
            else:
                l[2][t] = l[2][t] + stepsize * (l[5][t] - l[3][t])
et = time.time()
print(et - st)
traci.close()
sys.stdout.flush()

def run_normal():
    step = 0
    for v in vehicles:
        addvehicle(v)
    totalelectricity = 0
    while traci.simulation.getMinExpectedNumber() > 0 and step < T:
        traci.simulationStep()

```

```

#GETS ELECTRICITY CONSUMPTION
vehicle_ids = traci.vehicle.getIDList()
for vehicle_id in vehicle_ids:
    totalelectricity += traci.vehicle.getElectricityConsumption(vehicle_id)
    step += 1

vehicle_ids = traci.vehicle.getIDList()

total_travel_time = 0
num_vehicles = len(vehicle_ids)

# calculate the total travel time for all vehicles
for vehicle_id in vehicle_ids:
    travel_time = traci.vehicle.getAccumulatedWaitingTime(vehicle_id)
    total_travel_time += travel_time

# calculate the average travel time per vehicle
avg_travel_time = total_travel_time / num_vehicles

'''print("Average travel time per vehicle:", avg_travel_time)'''
print("NORMAL ELECTRICITY CONSUMPTION")
return (totalelectricity/num_vehicles)
traci.close()
sys.stdout.flush()

# main entry point
if __name__ == "__main__":
    options = get_options()

    # check binary
    if options.nogui:
        sumoBinary = checkBinary('sumo')
    else:
        sumoBinary = checkBinary('sumo-gui')

    sumoBinary = checkBinary('sumo')

# traci starts sumo as a subprocess and then this script connects and runs

for i in range(n):
    traci.start([sumoBinary, "-c", "demo.sumocfg",
                  "--tripinfo-output", "tripinfo.xml"])
    run(True)

```

```
hugereset()
for i in range(n):
    traci.start([sumoBinary, "-c", "demo.sumocfg",
                  "--tripinfo-output", "tripinfo.xml"])
    run(False)

hugereset()
traci.start([sumoBinary, "-c", "demo.sumocfg",
              "--tripinfo-output", "tripinfo.xml"])
print([CONSUMPTIONold, CONSUMPTIONour, run_normal()])
```